Krish — CPU Reference Solver

Wave Sim · ECE 379K · Checkout study guide (Fri Nov 13) · everything you need to explain your portion

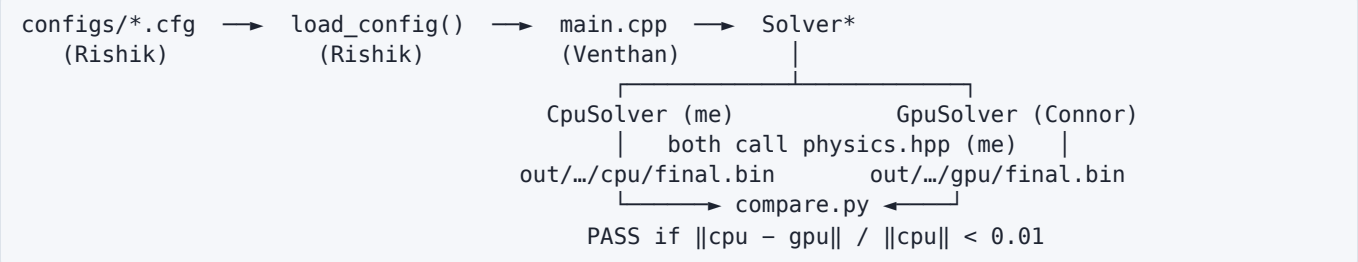
> **One-sentence version.** I wrote the single-core CPU simulation of the 3D wave equation. It is the "answer key": the GPU is graded by how closely its output matches mine (relative error < 1%). It calls the shared math in `physics.hpp`, passes 4 sanity checks, and reproduces all 4 reference numbers to 4 digits.

1. What I own and where it is

File	What it is
`include/physics.hpp`	Contract header (mine). The **only** place the math lives: `idx()`, `source_value()`, `update_cell()`. CPU and GPU both call it.
`include/solver.hpp`	Contract header (shared with Connor). The `Solver` interface both backends implement.
`src/cpu_solver.cpp`	**My main code.** `CpuSolver`: init / step / copy_field_to_host. ~100 lines.
`tools/test_cpu.cpp`	The 4 sanity checks + reference table. Run with `make test-cpu`.
`docs/PHYSICS.md`	Half-page doc: the equation, the CFL limit, why the walls reflect.

Branch: `claude/kind-bell-inh2pd`, two commits: (1) skeleton = the 4 contract headers copied from the plan appendix + placeholder stubs for `config.cpp`, `main.cpp`, `output.cpp`, `gpu_stub.cpp` so `make cpu` builds; (2) my work. The stubs belong to Rishik/Venthan/Connor and get replaced by their real files — **I only claim the files above.**

2. Where my piece fits



`main.cpp` only holds a `Solver*`; it never knows which backend is running. Because the formula lives in one header that both backends call, **the CPU and GPU can only differ in how they loop, never in what they compute.** That is the design decision that makes the GPU checkable.

3. The physics in plain words

We simulate sound pressure *p* in a closed box. The 3D wave equation says: the acceleration of the pressure at a point is proportional to how much it differs from its neighbours.

$$\partial^2 p/\partial t^2 = c^2\,\nabla^2 p + \text{sources} \qquad c = 343\ \text{m/s (sound in air)}$$

We put it on a grid (spacing `dx` = 1 cm) and step it in time (step `dt`). Two approximations:

- **Space — 7-point Laplacian.** $\nabla^2 p \approx$ (sum of the 6 neighbours − 6·centre) / dx^2. Each cell looks at left/right, front/back, up/down.
- **Time — leapfrog.** $\partial^2 p/\partial t^2 \approx (p_{next} - 2p_{cur} + p_{prev}) / dt^2$. Solving for p_{next}:

```
p_next = 2·p_cur − p_prev + r2 · lap        r2 = (c·dt/dx)² = cfl²
```

That is exactly `update_cell()`. It needs the **current and previous** field, which is why there are **three buffers**. `dt` is not chosen directly: we choose `cfl = c·dt/dx = 0.5`, so `dt = cfl·dx/c ≈ 14.6 µs`.

physics.hpp, line by line

```
HD inline size_t idx(int i,int j,int k,int nx,int ny){ return ((size_t)k*ny + j)*nx + i; }
   // 3D → 1D. x is fastest: neighbours in x are 1 apart, in y nx apart, in z nx*ny apart.
   // size_t so 512³ indices don't overflow int.

HD inline float source_value(const Source& s, float t)
   // Ricker:  amp·(1 − 2a)·e^(−a), a = (π f (t − 1/f))²   → one zero-mean pulse peaking at t0 =
1/f
   // Sine:    amp·sin(2π f t)                             → continuous tone

HD inline float update_cell(cur, prev, i,j,k, nx,ny, r2)
   lap = (cur[s-1]+cur[s+1]) + (cur[s-sy]+cur[s+sy]) + (cur[s-sz]+cur[s+sz]) - 6*c0;
   return 2*c0 - prev[s] + r2*lap;
   // Fixed summation order (x pair, y pair, z pair) so CPU and GPU round floats the same way.

#ifdef __CUDACC__  #define HD __host__ __device__
   // HD lets the SAME function compile for CPU (g++) and GPU (nvcc).
```

4. My code: `src/cpu_solver.cpp`

init(cfg)

```
cfg_ = cfg;
buf_a_.assign(n, 0.0f);  buf_b_.assign(n, 0.0f);  buf_c_.assign(n, 0.0f);   // n = nx*ny*nz, all
zero
prev_ = buf_a_.data();  cur_ = buf_b_.data();  next_ = buf_c_.data();
n_ = 0;
```

Three `std::vector<float>` own the memory; three raw pointers say which buffer currently plays which role. Field at step 0 is zero everywhere.

step() — the 5 rules, identical on GPU

```
// 1. Interior update (k outer, j, i inner → walks memory contiguously)
for k in 1..nz-2: for j in 1..ny-2: for i in 1..nx-2:
    next_[idx(i,j,k)] = update_cell(cur_, prev_, i,j,k, nx,ny, r2);
// 2. Walls: index 0 and N-1 are never written → stay 0
// 3. Sources AFTER the update, at the time of the NEW field
t = (n_+1)*dt;   for each src: next_[idx(src)] += source_value(src, t);
// 4. Rotate pointers (no copying)
old = prev_;  prev_ = cur_;  cur_ = next_;  next_ = old;
// 5. Count
++n_;
```

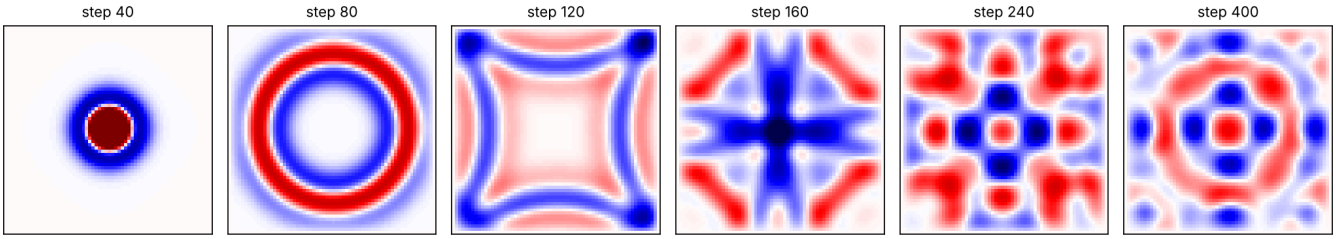
- **Why i innermost?** x is the fastest index, so consecutive `i` are consecutive addresses → cache-friendly. This is the "fair" one-core baseline.
- **Why `+=` for sources?** Two sources could sit on the same cell; also it adds on top of the wave already there.
- **Why `(n+1)*dt`?** We are computing the field at step n+1, so the source is evaluated at that time. The GPU must use the same time or results drift.
- **Why is reusing old prev as next safe?** Every interior cell of it is overwritten in step 1, and its walls are still 0 because no buffer ever has its walls written.
- **Why swap pointers?** Copying 3 × 2M floats per step would cost more than the update itself. Swapping is O(1).

copy_field_to_host(out)

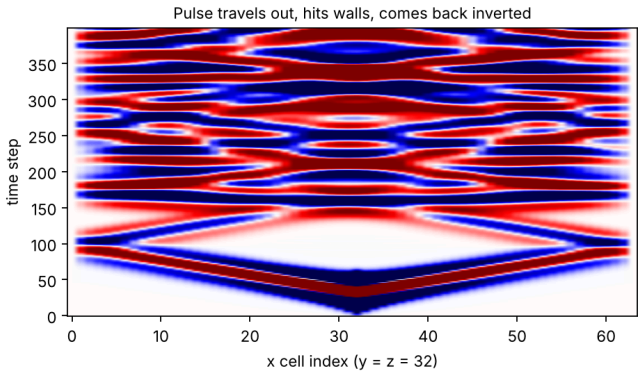
```
out.assign(cur_, cur_ + cfg_.n_cells());   // after the rotate, cur_ is the newest field
```

`sync()` is the default no-op (CPU work is already done when `step()` returns). `name()` returns `"cpu"`. `make_cpu_solver()` is the factory `main.cpp` calls. Single thread, no OpenMP — on purpose: the proposal's 5× target is against **one** core.

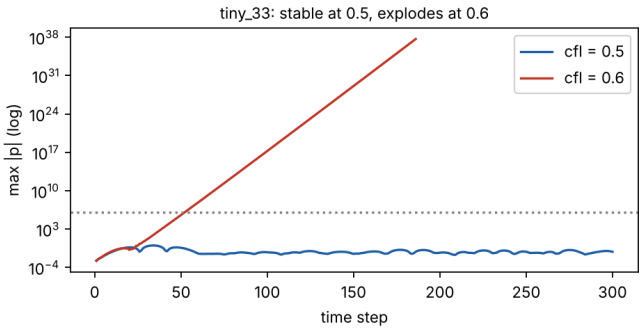
5. What it looks like (real output from my solver)



one_source_64, middle z-slice. Ricker pulse expands as a sphere (40–80), hits the walls (120), reflections cross and interfere (160+). Red = +p, blue = −p.



Line through the centre over time. The V is the pulse travelling at c; at the walls (x=0, 63) it bounces back with flipped colour = sign flip.



Log scale. cfl 0.5 stays bounded forever; cfl 0.6 grows exponentially (past 1e6 at step ~52, ~1e38 by 190).

6. The three concepts they'll ask about

CFL limit: cfl ≤ 1/√3 ≈ 0.577

`cfl = c·dt/dx` = how many cells the real wave moves per step. The stencil only lets information move one cell per axis per step. If the real wave moves faster than the grid can "see", the highest-frequency (checkerboard) error mode gets multiplied by a factor > 1 every step and grows exponentially. The √3 comes from 3 dimensions: the limit in 1D is 1, in d dimensions it is 1/√d (the worst case is the checkerboard mode whose Laplacian is $-4 \cdot d \cdot p/dx^2$; leapfrog stays stable while $r2 \cdot 4d \le 4$, i.e. $cfl^2 \le 1/d$). We use 0.5 for margin. Rishik's config rejects cfl > 0.577; check (d) proves why.

Why the walls reflect (with a sign flip)

The wall cells are never written, so p = 0 there forever (a "pressure-release"/Dirichlet boundary). For the wall to stay at 0 when a positive pulse arrives, a negative pulse must come back to cancel it — so it reflects **inverted**. No energy leaves the box, so the field never dies out (it just keeps sloshing — check (c)). Absorbing walls are explicitly out of scope this week.

Why float32 and why "within 1%" not "exactly equal"

GPU compilers may fuse `a*b+c` into one FMA instruction that rounds once instead of twice, and `expf/sinf` differ slightly between CPU libm and CUDA. So bits can differ; the fixed summation order keeps it tiny. The pass rule is ‖cpu − gpu‖ / ‖cpu‖ < 0.01 over `final.bin`. If Connor ever mismatches: build with `-fmad=false` and compare with `--steps 1`, then 2, then 3 to find the first bad step.

7. The 4 sanity checks (`tools/test_cpu.cpp`)

Configs are built directly in C++, so the tests don't depend on the config parser and could run on day one.

Check	Setup	Pass condition	What it proves	Result
(a) no sources	33^3, 100 steps, no sources	every cell exactly 0	nothing appears from nothing; no garbage memory	0 nonzero
(b) symmetry	tiny_33, source at centre (16,16,16)	$\|p(16+d)-p(16-d)\|$ and x↔y, x↔z swaps < 1e-5·max|p|	indexing & stencil treat all directions the same	mirror 0, swap 6e-8 (tol 5e-7)
(c) stable	tiny_33, cfl 0.5, 1000 steps	always finite; peak in 501–1000 ≤ peak in 1–500	scheme is stable below CFL	0.666 → 0.186
(d) unstable	same, cfl 0.6	NaN/inf or max|p| > 1e6	the CFL limit is real, so (c) means something	blew up at step 52

Why is mirror symmetry *exactly* 0 but axis-swap 6e-8? Mirroring only swaps the two terms inside a pair (a+b = b+a exactly in floats). Swapping axes changes which pair is added first, and float addition isn't associative — so tiny rounding differences. That's normal and well under the tolerance.

Reference numbers (max |p| after the last step) — all match

Config	Grid	Steps	Sources (Ricker 2000 Hz, amp 1)	Expected	My CPU	Time*
tiny_33	33^3	200	(16,16,16)	0.04977	0.04977	7 ms
one_source_64	64^3	1000	(32,32,32)	0.02026	0.02026	312 ms
two_source_64	64^3	1000	(20,32,32), (44,32,32)	0.03303	0.03303	336 ms
bench_128	128^3	1000	(64,64,64)	0.02129	0.02129	2.6 s

*Measured on a cloud dev container, not Lonestar6. The official 1-core baseline is whatever `bench.sh` records on LS6 (pinned with `taskset -c 0`). Rough sense: $128^3 \times 1000$ = 2.1 billion cell updates in ~2.6 s ≈ 0.8 G updates/s on one core.

8. Commands for the demo

```
make cpu
./wavesim configs/tiny_33.cfg --backend cpu      # prints max |p| = 0.04977
make test-cpu                                     # 4 checks + tiny_33 reference → ALL CHECKS
PASSED
./test_cpu --all-refs                             # full reference table (~3 s)
```

Expected output of `./test_cpu --all-refs`:

```
[PASS] (a) no sources -> all zero         nonzero cells after 100 steps: 0
[PASS] (b) tiny_33 mirror symmetry        max diff 0  (tol 4.98e-07)
[PASS] (b) tiny_33 axis-swap symmetry     max diff 6.33e-08  (tol 4.98e-07)
[PASS] (c) cfl 0.5, 1000 steps stable     peak steps 1-500 0.6658, peak 501-1000 0.1858
[PASS] (d) cfl 0.6 blows up               max|p| > 1e6 (or NaN) at step 52
[PASS] (ref) tiny_33                      max|p| = 0.04977  expected 0.04977
[PASS] (ref) one_source_64                max|p| = 0.02026  expected 0.02026
[PASS] (ref) two_source_64                max|p| = 0.03303  expected 0.03303
[PASS] (ref) bench_128                    max|p| = 0.02129  expected 0.02129
ALL CHECKS PASSED (0 failures)
```

> **Note on the placeholder stubs.** Until Rishik's parser lands, `load_config()` ignores the file and returns tiny_33, so `./wavesim <anything>` runs tiny_33. Once the real `config.cpp`, `main.cpp`, `output.cpp` are merged, the same `CpuSolver` works unchanged with every config and writes `out/<cfg>/cpu/final.bin`. If the team skeleton is pushed separately, take only my files from this branch.

9. Likely checkout questions — with answers

Q: Why do you need a CPU version at all if the project is about GPUs?
Two reasons: it's the correctness reference (simple, single-threaded, easy to reason about), and it's the 1-core speed baseline the 5× target is measured against.

Q: How do you know your CPU answer is right?
Four independent checks (zero, symmetry, stability, instability) plus reproducing the reference max|p| table to 4 digits. The pictures also show the physically expected behaviour: a spherical wavefront moving at c and reflecting inverted.

Q: Why three buffers?
Leapfrog needs p at n−1 and n to compute n+1. You can't update in place because neighbours still need the old value of the cell you just overwrote.

Q: Why don't you update the boundary cells?
They're the walls. Keeping them at 0 gives a reflecting boundary and also means the stencil never reads out of bounds (interior cells 1..N−2 only ever touch 0..N−1).

Q: What's the time step?
dt = cfl·dx/c = 0.5 × 0.01 / 343 ≈ 14.6 µs. 1000 steps ≈ 14.6 ms of simulated time; sound crosses the 64-cell box (0.63 m) in about 1.8 ms ≈ 126 steps — you can see the wall hit at step ~100–120 in the pictures.

Q: What is a Ricker wavelet?
The negative second derivative of a Gaussian: a short pulse with zero average, so it doesn't leave a permanent DC offset. Peaks at t0 = 1/f = 0.5 ms (~34 steps), basically over by ~70 steps.

Q: Why might CPU and GPU differ slightly?
Float rounding: FMA fusion on the GPU, different `expf/sinf` implementations, and any change in summation order. That's why the math is in one header with a fixed order, and why the pass rule is relative error < 1%, not bit-exact.

Q: Memory for 128^3?
2,097,152 cells × 4 bytes × 3 buffers ≈ 25 MB. Even 512^3 (the config max) is ~1.6 GB — fits on an A100.

Q: Why is the loop order k, j, i?
Layout is x-fastest (`idx = (k·ny + j)·nx + i`), so i innermost means sequential memory access. The GPU mirrors this: threadIdx.x → i, so a warp reads consecutive addresses (coalesced).

Q: Why is it single-threaded?
Deliberately: the proposal's speedup target is vs. one CPU core. Multithreading is listed under "not this week".

Q: What would you do if the GPU didn't match?
Run both with `--steps 1`, 2, 3 and diff `final.bin` to find the first step that differs; build GPU with `-fmad=false` to rule out FMA; check the source time is `(n+1)*dt` and walls aren't written.

10. My week plan status

#	Task	Status
1	Walk team through physics.hpp; freeze headers	Headers written exactly per the appendix — use section 3 of this guide for the walkthrough
2	Fill in cpu_solver.cpp; tiny_33 prints 0.04977	✅ Done
3	tools/test_cpu.cpp with 4 checks	✅ All 4 PASS (`make test-cpu`)
4	Run all configs, match reference table, post numbers	✅ All 4 reference numbers match (`./test_cpu --all-refs`); re-run through `./wavesim` once Rishik's configs land
5	Pair with Connor; write docs/PHYSICS.md	✅ Doc written; pairing happens at integration Thu